МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 3

Тема: **Структурные паттерны: Адаптер, Мост**

Дисциплина: **«Паттерны проектирования»**

Выполнила:

Студентка группы ПИЖ-б-о-23-2
направление подготовки 09.03.04
«Программная инженерия»

Силина Оксана Романовна

Проверил:

Ассистент ДЦРСИЭ

Белик А. И.

Ставрополь 2026

Цель работы: изучить принцип работы структурных паттернов проектирования «Адаптер» и «Мост» и реализовать прикладную задачу расчёта траектории движения тела с использованием данных паттернов.

Задача: разработать приложение для расчёта параметров движения тела, брошенного под углом к горизонту, с использованием двух структурных паттернов проектирования.

Выбранные паттерны:

- Адаптер — для интеграции сторонней библиотеки расчётов (например, библиотеки math или внешнего физического модуля) в интерфейс приложения;
- Мост — для разделения абстракции расчёта траектории и конкретной реализации модели движения (без сопротивления/с сопротивлением воздуха).

Индивидуальный вариант: 3. Разработка приложения «Траектория движения»

Репозиторий: https://github.com/tsukuri1/patterns

1. Описание паттернов

1.1. Адаптер (Adapter)

Адаптер — это структурный паттерн проектирования, который позволяет объектам с несовместимыми интерфейсами работать вместе. Адаптер выступает посредником, преобразуя вызовы одного интерфейса в вызовы другого.

Применение в задаче:

В приложении есть собственный интерфейс расчёта TrajectoryCalculator с методами flight_time(), max_height(), flight_range(). Сторонняя библиотека (например, модуль math или внешний физический модуль ExternalPhysicsLib) имеет другой интерфейс. Класс PhysicsLibAdapter реализует интерфейс TrajectoryCalculator, но внутри делегирует вызовы сторонней библиотеке, преобразуя параметры.

1.2. Мост (Bridge)

Мост — это структурный паттерн проектирования, который разделяет абстракцию и её реализацию так, чтобы они могли изменяться независимо. Вместо жёсткой иерархии наследования используется композиция: абстракция содержит ссылку на объект реализации.

В паттерне «Мост» абстракция Trajectory описывает способ расчёта траектории, а реализация MotionModel — физическую модель движения. В приложении реализованы два способа расчёта (AnalyticalTrajectory — по готовым формулам и NumericTrajectory — по массиву точек) и две модели движения (IdealModel — без сопротивления и DragModel — с сопротивлением). Наличие двух способов расчёта демонстрирует главное преимущество паттерна: абстракция и реализация могут изменяться независимо, а любая комбинация «способ × модель» работает без изменения кода.

Применение в задаче:

Абстракция Trajectory (способ расчёта траектории) отделена от реализации MotionModel (модель движения: идеальная или с сопротивлением). Класс Trajectory содержит ссылку на MotionModel и делегирует ему вычисления. Благодаря этому можно независимо добавлять новые способы расчёта (например, численное интегрирование) и новые модели движения, не меняя друг друга.

2. Структура приложения

Приложение состоит из следующих компонентов:

Компонент	Назначение
TrajectoryCalculator	Интерфейс расчёта (целевой интерфейс для Адаптера)
ExternalPhysicsLib	Сторонняя библиотека с несовместимым интерфейсом

PhysicsLibAdapter	Адаптер, приводящий ExternalPhysicsLib к TrajectoryCalculator
MotionModel	Реализация модели движения (часть Моста)
IdealModel	Идеальная модель движения
DragModel	Модель с сопротивлением воздуха
Trajectory	Абстракция расчёта траектории (часть Моста)
AnalyticalTrajectory	Аналитический способ расчёта
NumericTrajectory	Численный способ расчёта
TrajectoryApp	Графический интерфейс приложения

3. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

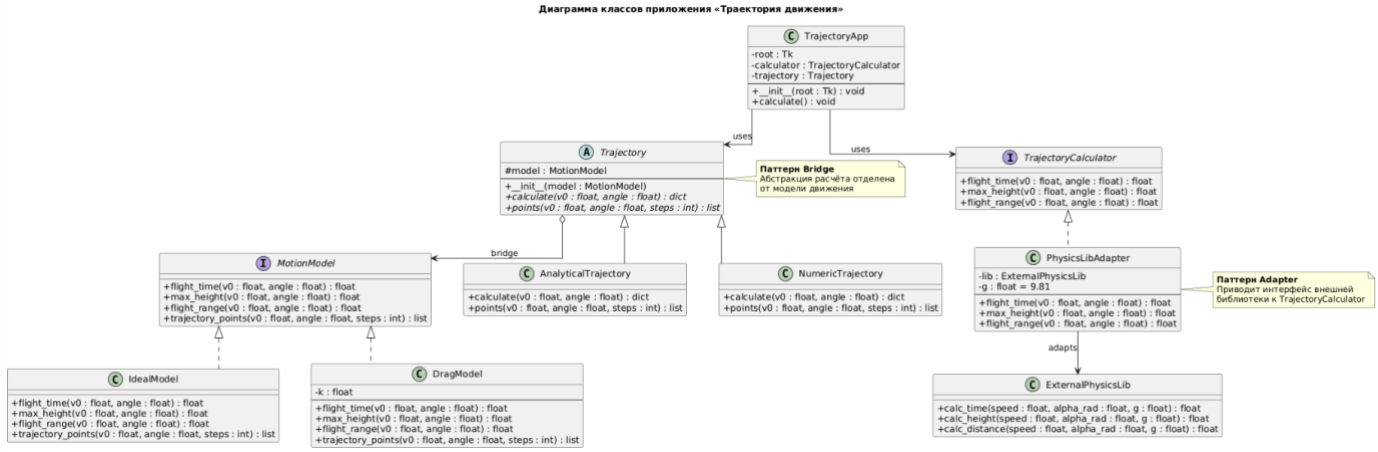


Рисунок 1 — Диаграмма классов приложения «Траектория движения»

Диаграмма содержит:

- Интерфейс TrajectoryCalculator, адаптер PhysicsLibAdapter и стороннюю библиотеку ExternalPhysicsLib (паттерн «Адаптер»);
- Абстракцию Trajectory, её наследников AnalyticalTrajectory, NumericTrajectory и реализации MotionModel (IdealModel, DragModel) (паттерн «Мост»);
- Класс TrajectoryApp — интерфейс приложения.

4. Реализация паттернов

4.1. Паттерн «Адаптер»

Внешняя библиотека имеет несовместимый интерфейс: она принимает угол в радианах и ускорение свободного падения как параметр, а методы называются иначе. Адаптер PhysicsLibAdapter реализует целевой интерфейс TrajectoryCalculator, внутри преобразуя градусы в радианы и делегируя вызовы библиотеке.

```
import math


class TrajectoryCalculator:
    """Целевой интерфейс расчёта."""
    def flight_time(self, v0, angle):
        raise NotImplementedError

    def max_height(self, v0, angle):
        raise NotImplementedError

    def flight_range(self, v0, angle):
        raise NotImplementedError


class ExternalPhysicsLib:
    """Сторонняя библиотека с несовместимым интерфейсом."""
    def calc_time(self, speed, alpha_rad, g):
        return 2 * speed * math.sin(alpha_rad) / g

    def calc_height(self, speed, alpha_rad, g):
        return (speed ** 2) * (math.sin(alpha_rad) ** 2) / (2 * g)

    def calc_distance(self, speed, alpha_rad, g):
        return (speed ** 2) * math.sin(2 * alpha_rad) / g


class PhysicsLibAdapter(TrajectoryCalculator):
    """Адаптер: приводит ExternalPhysicsLib к TrajectoryCalculator."""
```

```
def __init__(self, lib: ExternalPhysicsLib, g=9.81):
    self.lib = lib
    self.g = g

def flight_time(self, v0, angle):
    return self.lib.calc_time(v0, math.radians(angle), self.g)

def max_height(self, v0, angle):
    return self.lib.calc_height(v0, math.radians(angle), self.g)

def flight_range(self, v0, angle):
    return self.lib.calc_distance(v0, math.radians(angle), self.g)
```

4.2 Паттерн «Мост»

Абстракция Trajectory содержит ссылку на реализацию MotionModel. Конкретные способы расчёта (AnalyticalTrajectory, NumericTrajectory) используют переданную модель, но не зависят от её конкретного типа. Модели (IdealModel, DragModel) реализуют физику движения и также могут изменяться независимо.

```
class MotionModel:
    """Реализация модели движения (часть Моста)."""
    def flight_time(self, v0, angle):
        raise NotImplementedError

    def max_height(self, v0, angle):
        raise NotImplementedError

    def flight_range(self, v0, angle):
        raise NotImplementedError

    def trajectory_points(self, v0, angle, steps):
        raise NotImplementedError
```

```
class IdealModel(MotionModel):
    """Идеальная модель (без сопротивления)."""
    def __init__(self, g=9.81):
        self.g = g

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / self.g

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / self.g

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts
```

```
class DragModel(MotionModel):
    """Модель с сопротивлением воздуха."""
    def __init__(self, g=9.81, k=0.1):
        self.g = g
        self.k = k

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / (self.g * (1 + self.k))

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g * (1 + self.k))

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / (self.g * (1 + self.k))

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t / (1 + self.k * t)
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(max(y, 0), 3)))
        return pts
```

```
class Trajectory:
    """Абстракция расчёта траектории (часть Моста)."""
    def __init__(self, model: MotionModel):
        self.model = model

    def calculate(self, v0, angle):
        raise NotImplementedError

    def points(self, v0, angle, steps):
        raise NotImplementedError


class AnalyticalTrajectory(Trajectory):
    """Аналитический способ расчёта."""
    def calculate(self, v0, angle):
        return {
            "time": self.model.flight_time(v0, angle),
            "height": self.model.max_height(v0, angle),
            "range": self.model.flight_range(v0, angle),
        }

    def points(self, v0, angle, steps):
        return self.model.trajectory_points(v0, angle, steps)


class NumericTrajectory(Trajectory):
    """Численный способ расчёта (демонстрация независимости абстракции)."""
    def calculate(self, v0, angle):
        pts = self.model.trajectory_points(v0, angle, 100)
```

```
        T = self.model.flight_time(v0, angle)
        H = max(p[1] for p in pts)
        L = max(p[0] for p in pts)
        return {"time": T, "height": H, "range": L}

    def points(self, v0, angle, steps):
        return self.model.trajectory_points(v0, angle, steps)
```

4.3. Графический интерфейс

Интерфейс позволяет выбрать способ расчёта (аналитический/численный) и модель движения (идеальная/с сопротивлением), а также продемонстрировать работу адаптера, переключившись на расчёт через стороннюю библиотеку. Это наглядно показывает, как оба паттерна работают в реальном приложении.

```
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt


class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        root.title("Траектория движения")

        self.calculator = PhysicsLibAdapter(ExternalPhysicsLib())
        self.trajectory = AnalyticalTrajectory(IdealModel())

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0, column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)
```

```
        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0, sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Модель:").grid(row=2, column=0, sticky="w")
        self.model_var = tk.StringVar(value="ideal")
        tk.Radiobutton(root, text="Идеальная", variable=self.model_var,
                       value="ideal", command=self.update_model).grid(row=2, column=1, sticky="w")
        tk.Radiobutton(root, text="С сопротивлением", variable=self.model_var,
                       value="drag", command=self.update_model).grid(row=3, column=1, sticky="w")

        tk.Label(root, text="Способ расчёта:").grid(row=4, column=0, sticky="w")
        self.method_var = tk.StringVar(value="analytical")
        tk.Radiobutton(root, text="Аналитический", variable=self.method_var,
                       value="analytical", command=self.update_method).grid(row=4, column=1, sticky="w")
        tk.Radiobutton(root, text="Численный", variable=self.method_var,
                       value="numeric", command=self.update_method).grid(row=5, column=1, sticky="w")

        tk.Button(root, text="Рассчитать",
                  command=self.calculate).grid(row=6, column=0, columnspan=2, pady=5)
        tk.Button(root, text="Проверить через адаптер",
                  command=self.calculate_via_adapter).grid(row=7, column=0, columnspan=2)

        self.result = tk.Text(root, width=50, height=10)
        self.result.grid(row=8, column=0, columnspan=2)

    def update_model(self):
        model = IdealModel() if self.model_var.get() == "ideal" else DragModel()
```

```
        self.trajectory.model = model

    def update_method(self):
        model = self.trajectory.model
        if self.method_var.get() == "analytical":
            self.trajectory = AnalyticalTrajectory(model)
        else:
            self.trajectory = NumericTrajectory(model)

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
        angle = float(self.angle_entry.get())
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        return v0, angle

    def calculate(self):
        try:
            v0, angle = self._read_inputs()
            res = self.trajectory.calculate(v0, angle)
            pts = self.trajectory.points(v0, angle, 50)

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Способ: {type(self.trajectory).__name__}\n"
                f"Модель: {type(self.trajectory.model).__name__}\n"
                f"Время полёта: {res['time']:.3f} с\n"
                f"Макс. высота: {res['height']:.3f} м\n"
                f"Дальность: {res['range']:.3f} м\n")

            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title(f"{type(self.trajectory).__name__} / "
                      f"{type(self.trajectory.model).__name__}")
            plt.xlabel("X, м")
            plt.ylabel("Y, м")
```

```
            plt.grid(True)
            plt.axis('equal')
            plt.show()
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))

    def calculate_via_adapter(self):
        """Демонстрация паттерна Адаптер."""
        try:
            v0, angle = self._read_inputs()
            T = self.calculator.flight_time(v0, angle)
            H = self.calculator.max_height(v0, angle)
            L = self.calculator.flight_range(v0, angle)
            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Расчёт через адаптер (ExternalPhysicsLib):\n"
                f"Время полёта: {T:.3f} с\n"
                f"Макс. высота: {H:.3f} м\n"
                f"Дальность: {L:.3f} м\n")
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
    root.mainloop()
```

5. Результаты работы программы

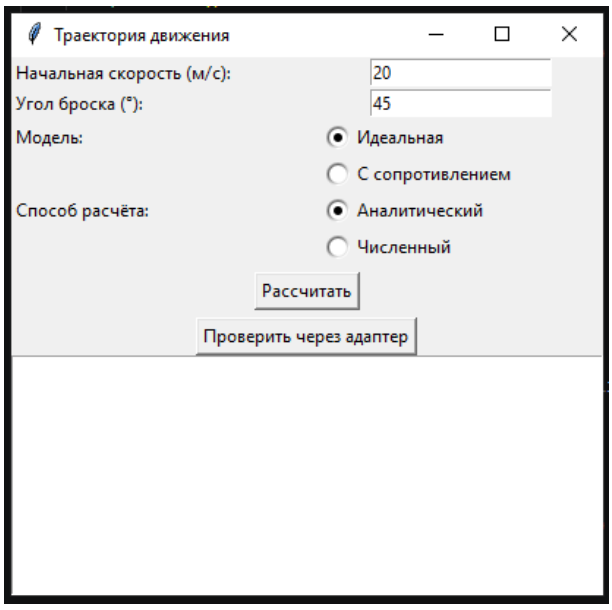


Рисунок 2 — Главное окно приложения

Пользователь вводит начальную скорость и угол броска, выбирает модель движения (идеальная/с сопротивлением) и способ расчёта (аналитический/численный).

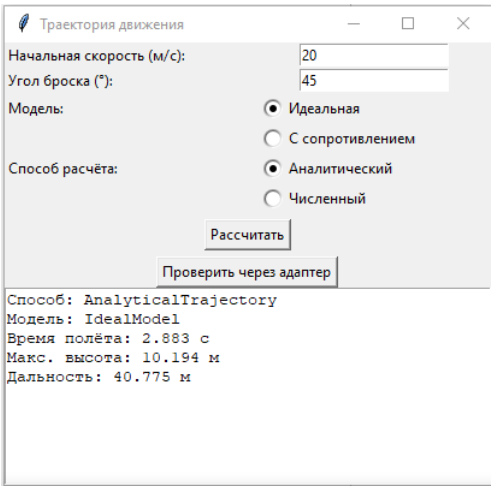


Рисунок 3 — Расчёт аналитическим способом с идеальной моделью

При v0 = 20 м/с, угол = 45°:

- Время полёта: 2.883 с
- Макс. высота: 10.194 м
- Дальность: 40.774 м

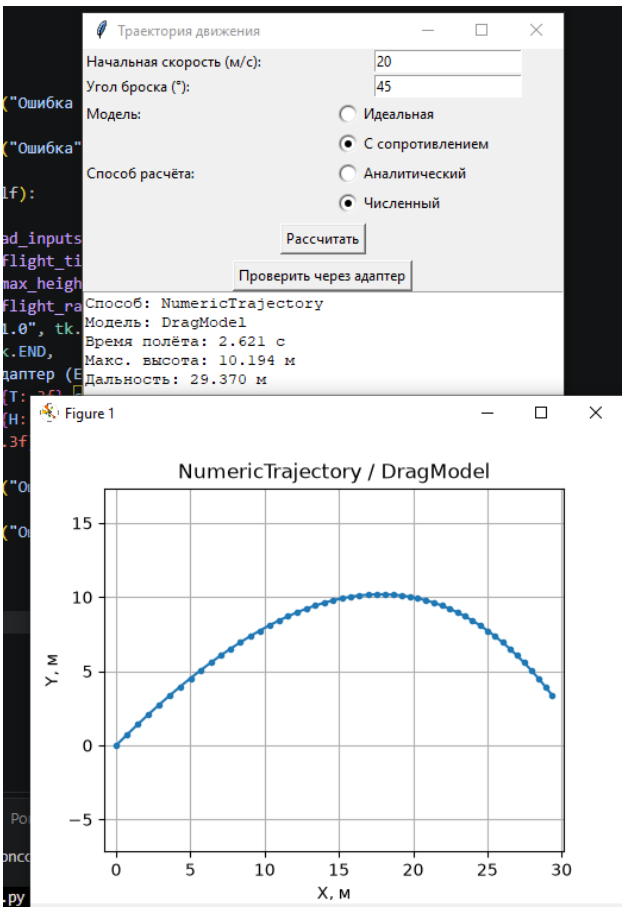


Рисунок 4 — Расчёт численным способом с моделью сопротивления

Значения дальности и высоты меньше за счёт сопротивления воздуха.

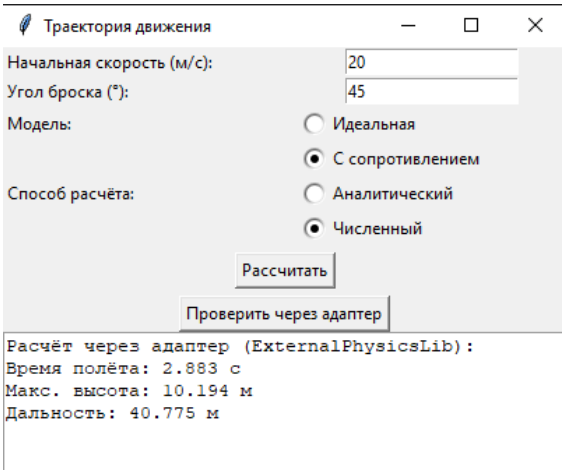


Рисунок 5 — Расчёт через адаптер

Кнопка «Проверить через адаптер» вызывает PhysicsLibAdapter, который делегирует расчёт внешней библиотеке ExternalPhysicsLib. Результаты

совпадают с аналитическим расчётом, что подтверждает корректность адаптера.

6. Исходный код

```
import math
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt


class TrajectoryCalculator:
    """Целевой интерфейс расчёта."""
    def flight_time(self, v0, angle):
        raise NotImplementedError

    def max_height(self, v0, angle):
        raise NotImplementedError

    def flight_range(self, v0, angle):
        raise NotImplementedError


class ExternalPhysicsLib:
    """Сторонняя библиотека с несовместимым интерфейсом."""
    def calc_time(self, speed, alpha_rad, g):
        return 2 * speed * math.sin(alpha_rad) / g

    def calc_height(self, speed, alpha_rad, g):
        return (speed ** 2) * (math.sin(alpha_rad) ** 2) / (2 * g)

    def calc_distance(self, speed, alpha_rad, g):
        return (speed ** 2) * math.sin(2 * alpha_rad) / g


class PhysicsLibAdapter(TrajectoryCalculator):
    """Адаптер: приводит ExternalPhysicsLib к TrajectoryCalculator."""
    def __init__(self, lib: ExternalPhysicsLib, g=9.81):
```

```
        self.lib = lib
        self.g = g

    def flight_time(self, v0, angle):
        return self.lib.calc_time(v0, math.radians(angle), self.g)

    def max_height(self, v0, angle):
        return self.lib.calc_height(v0, math.radians(angle), self.g)

    def flight_range(self, v0, angle):
        return self.lib.calc_distance(v0, math.radians(angle), self.g)


class MotionModel:
    """Реализация модели движения (часть Моста)."""
    def flight_time(self, v0, angle):
        raise NotImplementedError

    def max_height(self, v0, angle):
        raise NotImplementedError

    def flight_range(self, v0, angle):
        raise NotImplementedError

    def trajectory_points(self, v0, angle, steps):
        raise NotImplementedError


class IdealModel(MotionModel):
    def __init__(self, g=9.81):
        self.g = g

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / self.g

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)
```

```
    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / self.g

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class DragModel(MotionModel):
    def __init__(self, g=9.81, k=0.1):
        self.g = g
        self.k = k

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / (self.g * (1 + self.k))

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g * (1 + self.k))

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / (self.g * (1 + self.k))

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
```

```
            t = T * i / steps
            x = v0 * math.cos(a) * t / (1 + self.k * t)
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(max(y, 0), 3)))
        return pts


class Trajectory:
    """Абстракция расчёта траектории (часть Моста)."""
    def __init__(self, model: MotionModel):
        self.model = model

    def calculate(self, v0, angle):
        raise NotImplementedError

    def points(self, v0, angle, steps):
        raise NotImplementedError


class AnalyticalTrajectory(Trajectory):
    def calculate(self, v0, angle):
        return {
            "time": self.model.flight_time(v0, angle),
            "height": self.model.max_height(v0, angle),
            "range": self.model.flight_range(v0, angle),
        }

    def points(self, v0, angle, steps):
        return self.model.trajectory_points(v0, angle, steps)


class NumericTrajectory(Trajectory):
    def calculate(self, v0, angle):
        pts = self.model.trajectory_points(v0, angle, 100)
        T = self.model.flight_time(v0, angle)
        H = max(p[1] for p in pts)
        L = max(p[0] for p in pts)
        return {"time": T, "height": H, "range": L}
```

```
    def points(self, v0, angle, steps):
        return self.model.trajectory_points(v0, angle, steps)


class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        root.title("Траектория движения")

        self.calculator = PhysicsLibAdapter(ExternalPhysicsLib())
        self.trajectory = AnalyticalTrajectory(IdealModel())

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Модель:").grid(row=2, column=0, sticky="w")
        self.model_var = tk.StringVar(value="ideal")
        tk.Radiobutton(root, text="Идеальная", variable=self.model_var,
                       value="ideal",       command=self.update_model).grid(row=2,
column=1, sticky="w")
        tk.Radiobutton(root,                text="С                сопротивлением",
variable=self.model_var,
                       value="drag",        command=self.update_model).grid(row=3,
column=1, sticky="w")

        tk.Label(root,      text="Способ      расчёта:").grid(row=4,      column=0,
sticky="w")
        self.method_var = tk.StringVar(value="analytical")
        tk.Radiobutton(root, text="Аналитический", variable=self.method_var,
```

```
                value="analytical",
command=self.update_method).grid(row=4, column=1, sticky="w")
        tk.Radiobutton(root, text="Численный", variable=self.method_var,
                value="numeric", command=self.update_method).grid(row=5,
column=1, sticky="w")

        tk.Button(root, text="Рассчитать",
                command=self.calculate).grid(row=6, column=0, columnspan=2,
pady=5)
        tk.Button(root, text="Проверить через адаптер",
                command=self.calculate_via_adapter).grid(row=7,      column=0,
columnspan=2)

        self.result = tk.Text(root, width=50, height=10)
        self.result.grid(row=8, column=0, columnspan=2)

    def update_model(self):
        model = IdealModel() if self.model_var.get() == "ideal" else
DragModel()
        self.trajectory.model = model

    def update_method(self):
        model = self.trajectory.model
        if self.method_var.get() == "analytical":
            self.trajectory = AnalyticalTrajectory(model)
        else:
            self.trajectory = NumericTrajectory(model)

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
        angle = float(self.angle_entry.get())
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        return v0, angle

    def calculate(self):
        try:
```

```
            v0, angle = self._read_inputs()
            res = self.trajectory.calculate(v0, angle)
            pts = self.trajectory.points(v0, angle, 50)

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Способ: {type(self.trajectory).__name__}\n"
                f"Модель: {type(self.trajectory.model).__name__}\n"
                f"Время полёта: {res['time']:.3f} с\n"
                f"Макс. высота: {res['height']:.3f} м\n"
                f"Дальность: {res['range']:.3f} м\n")

            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title(f"{type(self.trajectory).__name__} / "
                      f"{type(self.trajectory.model).__name__}")
            plt.xlabel("X, м")
            plt.ylabel("Y, м")
            plt.grid(True)
            plt.axis('equal')
            plt.show()
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))

    def calculate_via_adapter(self):
        try:
            v0, angle = self._read_inputs()
            T = self.calculator.flight_time(v0, angle)
            H = self.calculator.max_height(v0, angle)
            L = self.calculator.flight_range(v0, angle)
            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Расчёт через адаптер (ExternalPhysicsLib):\n"
                f"Время полёта: {T:.3f} с\n"
                f"Макс. высота: {H:.3f} м\n"
                f"Дальность: {L:.3f} м\n")
```

```
            except ValueError as e:
                messagebox.showerror("Ошибка ввода", str(e))
            except Exception as e:
                messagebox.showerror("Ошибка", str(e))


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
    root.mainloop()
```

Вывод:

В ходе выполнения лабораторной работы было разработано приложение «Траектория движения», реализующее расчёт параметров движения тела, брошенного под углом к горизонту.

В приложении применены два структурных паттерна проектирования:

- Адаптер — для интеграции сторонней библиотеки ExternalPhysicsLib в интерфейс TrajectoryCalculator. Класс PhysicsLibAdapter преобразует вызовы и параметры, благодаря чему приложение может использовать внешний код без изменения собственной архитектуры.
- Мост — для разделения абстракции расчёта траектории (Trajectory) и реализации модели движения (MotionModel). Это позволяет независимо добавлять новые способы расчёта (аналитический, численный) и новые модели движения (идеальная, с сопротивлением).

Таким образом, была изучена практическая реализация двух структурных паттернов и получено работоспособное приложение с графическим интерфейсом для расчёта и визуализации траектории движения тела.